AI Assisted Coding

Assignment –16.3

Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

• List all appointments for a specific doctor.

• Retrieve patient history by patient ID.

• Count total patients treated by each doctor.

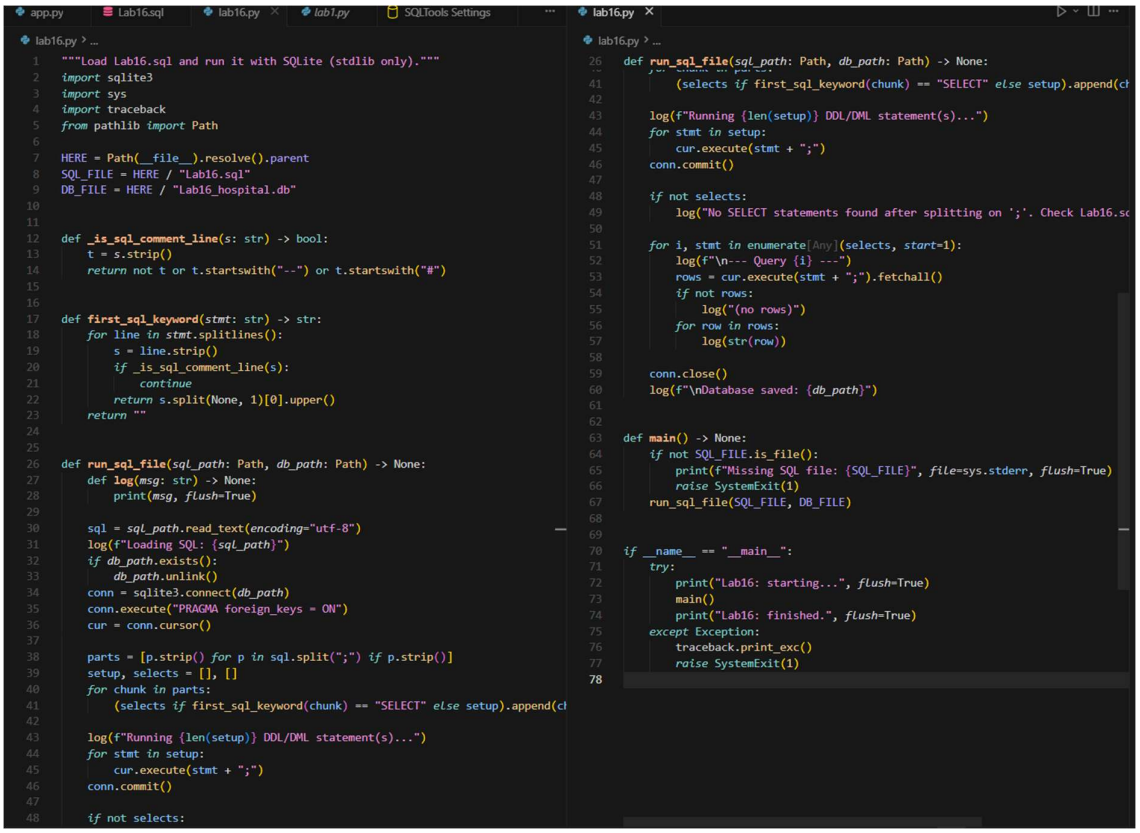
Expected Output:

• Normalized schema and SQL queries with joins.

Lab16.sql

Run on active connection | Select block

```sql
-- Design a normalized relational database schema for a Hospital Management System using SQL.
--
-- Requirements:
-- - Tables: Doctors, Patients, Appointments
-- - Include primary keys, foreign keys, constraints (unique IDs, valid dates)
-- - Ensure normalization (up to 3NF)
--
-- Generate SQL queries:
-- 1. List all appointments for a specific doctor
-- 2. Retrieve patient history by patient ID
-- 3. Count total patients treated by each doctor
--
-- - Use proper JOINs
-- - Include sample INSERT data
-- - Add comments explaining schema and queries
--
-- Output complete SQL script.
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    specialization VARCHAR(255) NOT NULL
);
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    date_of_birth DATE NOT NULL
);
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
INSERT INTO Doctors (doctor_id, name, specialization) VALUES
    (1, 'Dr. John Doe', 'Cardiologist'),
    (2, 'Dr. Jane Smith', 'Pediatrician'),
    (3, 'Dr. Michael Brown', 'Dermatologist');
INSERT INTO Patients (patient_id, name, date_of_birth) VALUES
    (1, 'John Doe', '1980-01-01'),
    (2, 'Jane Smith', '1990-05-15'),
    (3, 'Michael Brown', '1985-07-20');
INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES
    (1, 1, 1, '2024-01-01'),
    (2, 2, 2, '2024-02-01'),
    (3, 3, 3, '2024-03-01');
-- List all appointments for a specific doctor
SELECT * FROM Appointments WHERE doctor_id = 1;
-- Retrieve patient history by patient ID
SELECT * FROM Patients WHERE patient_id = 1;
-- Count total patients treated by each doctor
SELECT doctor_id, COUNT(*) AS total_patients FROM Appointments GROUP BY doctor_id;
```



```python
"""Load Lab16.sql and run it with SQLite (stdlib only)."""
import sqlite3
import sys
import traceback
from pathlib import Path

HERE = Path(__file__).resolve().parent
SQL_FILE = HERE / "Lab16.sql"
DB_FILE = HERE / "Lab16_hospital.db"


def _is_sql_comment_line(s: str) -> bool:
    t = s.strip()
    return not t or t.startswith("--") or t.startswith("#")


def first_sql_keyword(stmt: str) -> str:
    for line in stmt.splitlines():
        s = line.strip()
        if _is_sql_comment_line(s):
            continue
        return s.split(None, 1)[0].upper()
    return ""


def run_sql_file(sql_path: Path, db_path: Path) -> None:
    def log(msg: str) -> None:
        print(msg, flush=True)

    sql = sql_path.read_text(encoding="utf-8")
    log(f"Loading SQL: {sql_path}")
    if db_path.exists():
        db_path.unlink()
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA foreign_keys = ON")
    cur = conn.cursor()

    parts = [p.strip() for p in sql.split(";") if p.strip()]
    setup, selects = [], []
    for chunk in parts:
        (selects if first_sql_keyword(chunk) == "SELECT" else setup).append(ch

    log(f"Running {len(setup)} DDL/DML statement(s)...")
    for stmt in setup:
        cur.execute(stmt + ";")
    conn.commit()

    if not selects:
```

```python
def run_sql_file(sql_path: Path, db_path: Path) -> None:
        (selects if first_sql_keyword(chunk) == "SELECT" else setup).append(ch

    log(f"Running {len(setup)} DDL/DML statement(s)...")
    for stmt in setup:
        cur.execute(stmt + ";")
    conn.commit()

    if not selects:
        log("No SELECT statements found after splitting on ';'. Check Lab16.sc

    for i, stmt in enumerate(selects, start=1):
        log(f"\n--- Query {i} ---")
        rows = cur.execute(stmt + ";").fetchall()
        if not rows:
            log("(no rows)")
        for row in rows:
            log(str(row))

    conn.close()
    log(f"\nDatabase saved: {db_path}")


def main() -> None:
    if not SQL_FILE.is_file():
        print(f"Missing SQL file: {SQL_FILE}", file=sys.stderr, flush=True)
        raise SystemExit(1)
    run_sql_file(SQL_FILE, DB_FILE)


if __name__ == "__main__":
    try:
        print("Lab16: starting...", flush=True)
        main()
        print("Lab16: finished.", flush=True)
    except Exception:
        traceback.print_exc()
        raise SystemExit(1)
```

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

• Retrieve all orders by a user.

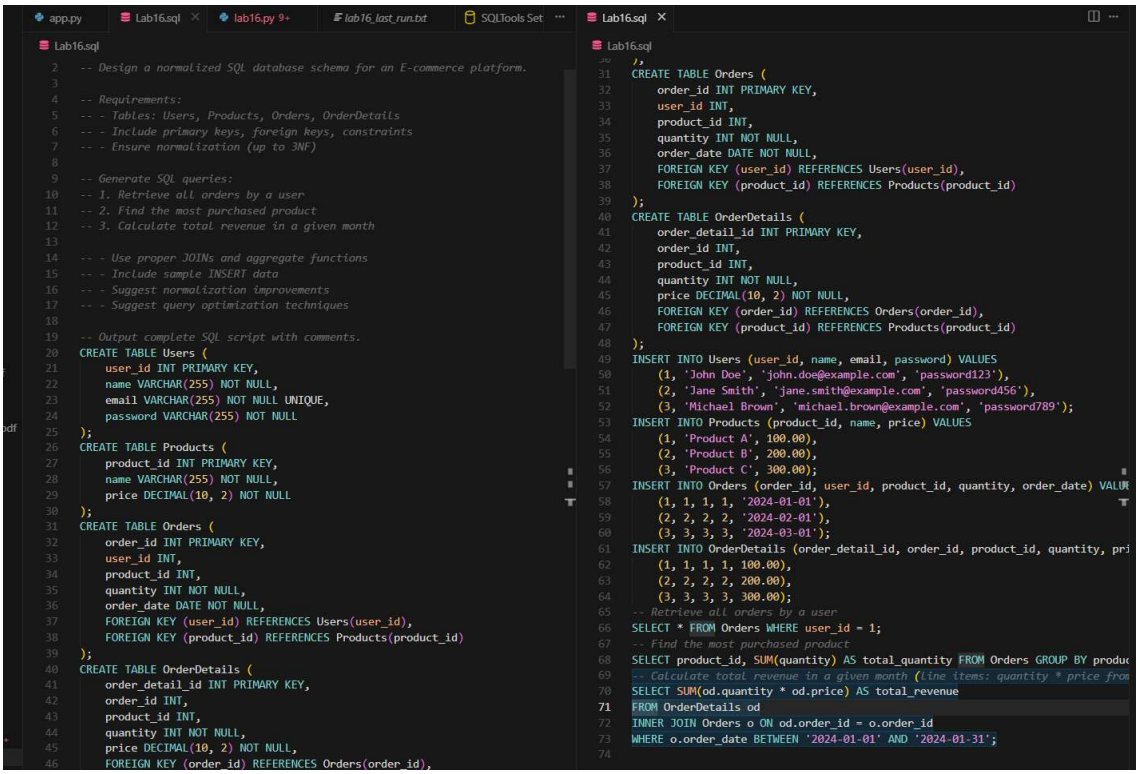
• Find the most purchased product.

• Calculate total revenue in a given month.

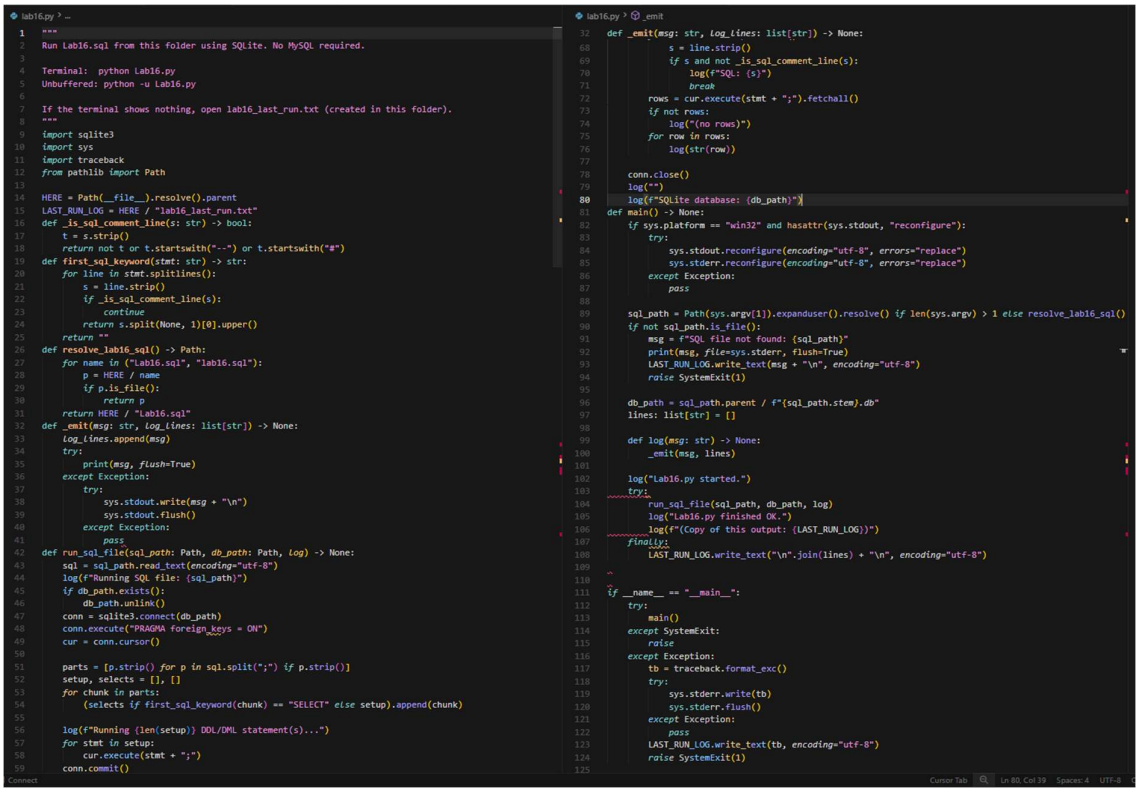
• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.



```sql
-- Design a normalized SQL database schema for an E-commerce platform.

-- Requirements:
-- - Tables: Users, Products, Orders, OrderDetails
-- - Include primary keys, foreign keys, constraints
-- - Ensure normalization (up to 3NF)

-- Generate SQL queries:
-- 1. Retrieve all orders by a user
-- 2. Find the most purchased product
-- 3. Calculate total revenue in a given month

-- - Use proper JOINs and aggregate functions
-- - Include sample INSERT data
-- - Suggest normalization improvements
-- - Suggest query optimization techniques

-- Output complete SQL script with comments.
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL
);
CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    price DECIMAL(10, 2) NOT NULL
);
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    product_id INT,
    quantity INT NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
CREATE TABLE OrderDetails (
    order_detail_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
INSERT INTO Users (user_id, name, email, password) VALUES
    (1, 'John Doe', 'john.doe@example.com', 'password123'),
    (2, 'Jane Smith', 'jane.smith@example.com', 'password456'),
    (3, 'Michael Brown', 'michael.brown@example.com', 'password789');
INSERT INTO Products (product_id, name, price) VALUES
    (1, 'Product A', 100.00),
    (2, 'Product B', 200.00),
    (3, 'Product C', 300.00);
INSERT INTO Orders (order_id, user_id, product_id, quantity, order_date) VALUE
    (1, 1, 1, 1, '2024-01-01'),
    (2, 2, 2, 2, '2024-02-01'),
    (3, 3, 3, 3, '2024-03-01');
INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity, pri
    (1, 1, 1, 1, 100.00),
    (2, 2, 2, 2, 200.00),
    (3, 3, 3, 3, 300.00);
-- Retrieve all orders by a user
SELECT * FROM Orders WHERE user_id = 1;
-- Find the most purchased product
SELECT product_id, SUM(quantity) AS total_quantity FROM Orders GROUP BY produc
-- Calculate total revenue in a given month (line items: quantity * price from
SELECT SUM(od.quantity * od.price) AS total_revenue
FROM OrderDetails od
INNER JOIN Orders o ON od.order_id = o.order_id
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-01-31';
```



```python
"""
Run Lab16.sql from this folder using SQLite. No MySQL required.

Terminal:  python Lab16.py
Unbuffered: python -u Lab16.py

If the terminal shows nothing, open lab16_last_run.txt (created in this folder).
"""
import sqlite3
import sys
import traceback
from pathlib import Path

HERE = Path(__file__).resolve().parent
LAST_RUN_LOG = HERE / "lab16_last_run.txt"
def _is_sql_comment_line(s: str) -> bool:
    t = s.strip()
    return not t or t.startswith("--") or t.startswith("#")
def first_sql_keyword(stmt: str) -> str:
    for line in stmt.splitlines():
        s = line.strip()
        if _is_sql_comment_line(s):
            continue
        return s.split(None, 1)[0].upper()
    return ""
def resolve_lab16_sql() -> Path:
    for name in ("Lab16.sql", "lab16.sql"):
        p = HERE / name
        if p.is_file():
            return p
    return HERE / "Lab16.sql"
def _emit(msg: str, log_lines: list[str]) -> None:
    log_lines.append(msg)
    try:
        print(msg, flush=True)
    except Exception:
        try:
            sys.stdout.write(msg + "\n")
            sys.stdout.flush()
        except Exception:
            pass
def run_sql_file(sql_path: Path, db_path: Path, log) -> None:
    sql = sql_path.read_text(encoding="utf-8")
    log(f"Running SQL file: {sql_path}")
    if db_path.exists():
        db_path.unlink()
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA foreign_keys = ON")
    cur = conn.cursor()

    parts = [p.strip() for p in sql.split(";") if p.strip()]
    setup, selects = [], []
    for chunk in parts:
        (selects if first_sql_keyword(chunk) == "SELECT" else setup).append(chunk)

    log(f"Running {len(setup)} DDL/DML statement(s)...")
    for stmt in setup:
        cur.execute(stmt + ";")
    conn.commit()
            s = line.strip()
            if s and not _is_sql_comment_line(s):
                log(f"SQL: {s}")
                break
        rows = cur.execute(stmt + ";").fetchall()
        if not rows:
            log("(no rows)")
        for row in rows:
            log(str(row))

    conn.close()
    log("")
    log(f"SQLite database: {db_path}")
def main() -> None:
    if sys.platform == "win32" and hasattr(sys.stdout, "reconfigure"):
        try:
            sys.stdout.reconfigure(encoding="utf-8", errors="replace")
            sys.stderr.reconfigure(encoding="utf-8", errors="replace")
        except Exception:
            pass

    sql_path = Path(sys.argv[1]).expanduser().resolve() if len(sys.argv) > 1 else resolve_lab16_sql()
    if not sql_path.is_file():
        msg = f"SQL file not found: {sql_path}"
        print(msg, file=sys.stderr, flush=True)
        LAST_RUN_LOG.write_text(msg + "\n", encoding="utf-8")
        raise SystemExit(1)

    db_path = sql_path.parent / f"{sql_path.stem}.db"
    lines: list[str] = []

    def log(msg: str) -> None:
        _emit(msg, lines)

    log("Lab16.py started.")
    try:
        run_sql_file(sql_path, db_path, log)
        log("Lab16.py finished OK.")
    except Exception:
        log(f"(Copy of this output: {LAST_RUN_LOG})")
        raise
    finally:
        LAST_RUN_LOG.write_text("\n".join(lines) + "\n", encoding="utf-8")

if __name__ == "__main__":
    try:
        main()
    except SystemExit:
        raise
    except Exception:
        tb = traceback.format_exc()
        try:
            sys.stderr.write(tb)
            sys.stderr.flush()
        except Exception:
            pass
        LAST_RUN_LOG.write_text(tb, encoding="utf-8")
        raise SystemExit(1)
```

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

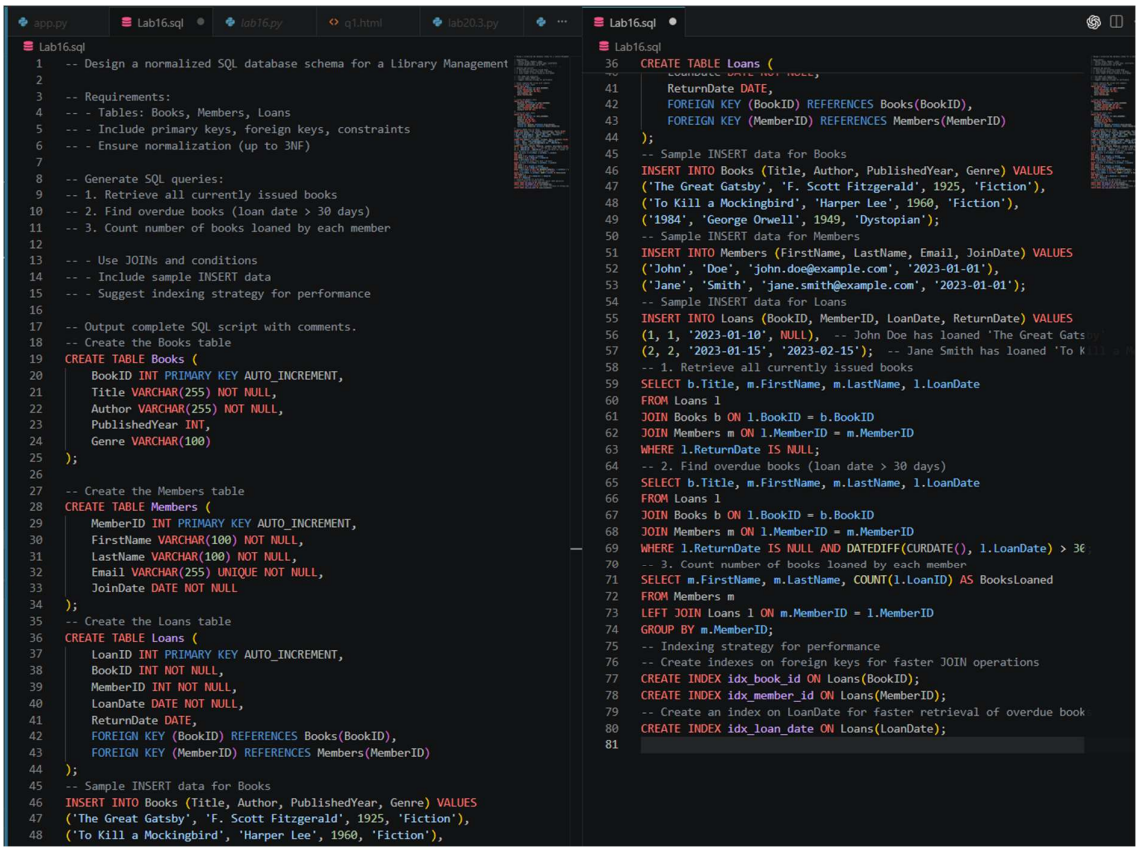
• Retrieve all books currently issued.

• Find overdue books (loan date > 30 days).

• Count number of books loaned by each member.

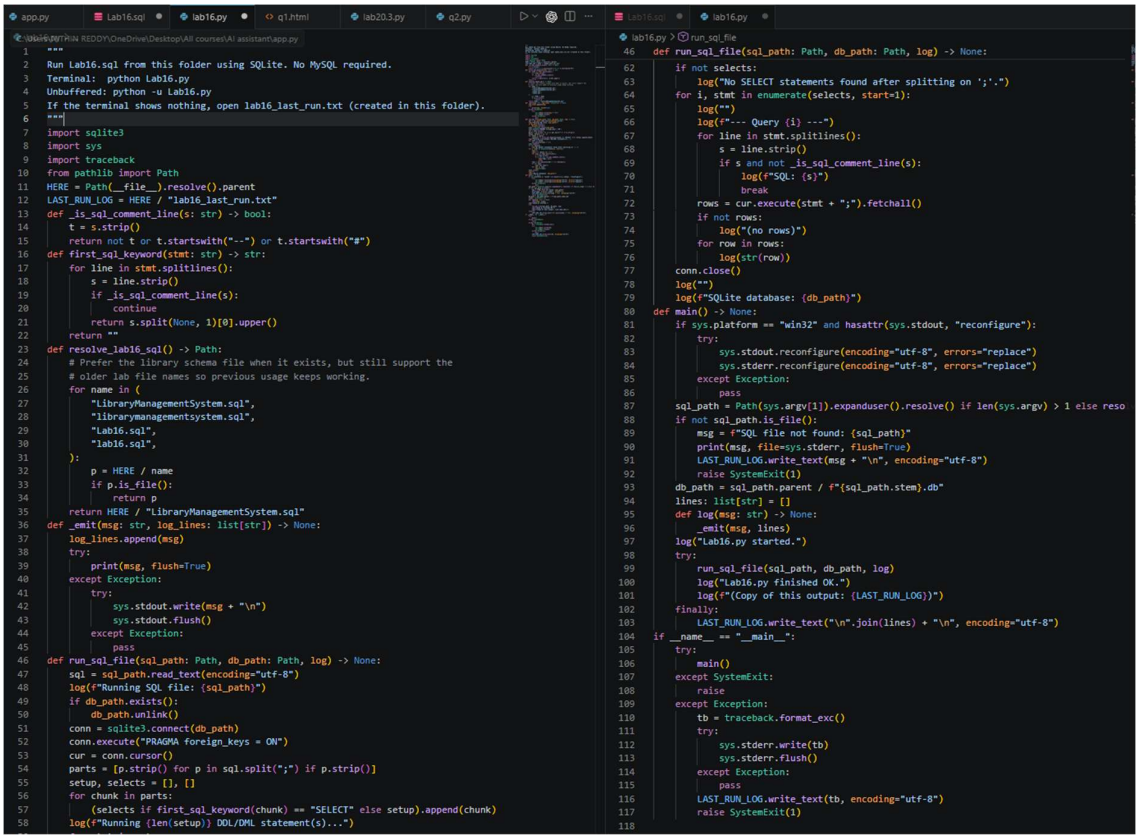
Expected Output:

• Schema + SQL queries demonstrating joins and conditions.



```sql
-- Design a normalized SQL database schema for a Library Management

-- Requirements:
-- - Tables: Books, Members, Loans
-- - Include primary keys, foreign keys, constraints
-- - Ensure normalization (up to 3NF)

-- Generate SQL queries:
-- 1. Retrieve all currently issued books
-- 2. Find overdue books (loan date > 30 days)
-- 3. Count number of books loaned by each member

-- - Use JOINs and conditions
-- - Include sample INSERT data
-- - Suggest indexing strategy for performance

-- Output complete SQL script with comments.
-- Create the Books table
CREATE TABLE Books (
    BookID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(255) NOT NULL,
    Author VARCHAR(255) NOT NULL,
    PublishedYear INT,
    Genre VARCHAR(100)
);

-- Create the Members table
CREATE TABLE Members (
    MemberID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(100) NOT NULL,
    LastName VARCHAR(100) NOT NULL,
    Email VARCHAR(255) UNIQUE NOT NULL,
    JoinDate DATE NOT NULL
);
-- Create the Loans table
CREATE TABLE Loans (
    LoanID INT PRIMARY KEY AUTO_INCREMENT,
    BookID INT NOT NULL,
    MemberID INT NOT NULL,
    LoanDate DATE NOT NULL,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Sample INSERT data for Books
INSERT INTO Books (Title, Author, PublishedYear, Genre) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925, 'Fiction'),
('To Kill a Mockingbird', 'Harper Lee', 1960, 'Fiction'),
```

```sql
CREATE TABLE Loans (
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Sample INSERT data for Books
INSERT INTO Books (Title, Author, PublishedYear, Genre) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 1925, 'Fiction'),
('To Kill a Mockingbird', 'Harper Lee', 1960, 'Fiction'),
('1984', 'George Orwell', 1949, 'Dystopian');
-- Sample INSERT data for Members
INSERT INTO Members (FirstName, LastName, Email, JoinDate) VALUES
('John', 'Doe', 'john.doe@example.com', '2023-01-01'),
('Jane', 'Smith', 'jane.smith@example.com', '2023-01-01');
-- Sample INSERT data for Loans
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate) VALUES
(1, 1, '2023-01-10', NULL),  -- John Doe has loaned 'The Great Gats
(2, 2, '2023-01-15', '2023-02-15');  -- Jane Smith has loaned 'To K
-- 1. Retrieve all currently issued books
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- 2. Find overdue books (loan date > 30 days)
SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND DATEDIFF(CURDATE(), l.LoanDate) > 3
-- 3. Count number of books loaned by each member
SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
-- Indexing strategy for performance
-- Create indexes on foreign keys for faster JOIN operations
CREATE INDEX idx_book_id ON Loans(BookID);
CREATE INDEX idx_member_id ON Loans(MemberID);
-- Create an index on LoanDate for faster retrieval of overdue book
CREATE INDEX idx_loan_date ON Loans(LoanDate);
```


app.py
Lab16.sql
lab16.py
q1.html
lab20.3.py
q2.py
C:\Users\VINTHAN REDDY\OneDrive\Desktop\All courses\AI assistant\app.py
"""
Run Lab16.sql from this folder using SQLite. No MySQL required.
Terminal: python Lab16.py
Unbuffered: python -u Lab16.py
If the terminal shows nothing, open lab16_last_run.txt (created in this folder).
"""
import sqlite3
import sys
import traceback
from pathlib import Path
HERE = Path(__file__).resolve().parent
LAST_RUN_LOG = HERE / "lab16_last_run.txt"
def _is_sql_comment_line(s: str) -> bool:
t = s.strip()
return not t or t.startswith("--") or t.startswith("#")
def first_sql_keyword(stmt: str) -> str:
for line in stmt.splitlines():
s = line.strip()
if _is_sql_comment_line(s):
continue
return s.split(None, 1)[0].upper()
return ""
def resolve_lab16_sql() -> Path:
# Prefer the library schema file when it exists, but still support the
# older lab file names so previous usage keeps working.
for name in (
"LibraryManagementSystem.sql",
"librarymanagementsystem.sql",
"Lab16.sql",
"lab16.sql",
):
p = HERE / name
if p.is_file():
return p
return HERE / "LibraryManagementSystem.sql"
def _emit(msg: str, log_lines: list[str]) -> None:
log_lines.append(msg)
try:
print(msg, flush=True)
except Exception:
try:
sys.stdout.write(msg + "\n")
sys.stdout.flush()
except Exception:
pass
def run_sql_file(sql_path: Path, db_path: Path, log) -> None:
sql = sql_path.read_text(encoding="utf-8")
log(f"Running SQL file: {sql_path}")
if db_path.exists():
db_path.unlink()
conn = sqlite3.connect(db_path)
conn.execute("PRAGMA foreign_keys = ON")
cur = conn.cursor()
parts = [p.strip() for p in sql.split(";") if p.strip()]
setup, selects = [], []
for chunk in parts:
(selects if first_sql_keyword(chunk) == "SELECT" else setup).append(chunk)
log(f"Running {len(setup)} DDL/DML statement(s)...")
lab16.py
run_sql_file
def run_sql_file(sql_path: Path, db_path: Path, log) -> None:
if not selects:
log("No SELECT statements found after splitting on ';'.")
for i, stmt in enumerate(selects, start=1):
log("")
log(f"--- Query {i} ---")
for line in stmt.splitlines():
s = line.strip()
if s and not _is_sql_comment_line(s):
log(f"SQL: {s}")
break
rows = cur.execute(stmt + ";").fetchall()
if not rows:
log("(no rows)")
for row in rows:
log(str(row))
conn.close()
log("")
log(f"SQLite database: {db_path}")
def main() -> None:
if sys.platform == "win32" and hasattr(sys.stdout, "reconfigure"):
try:
sys.stdout.reconfigure(encoding="utf-8", errors="replace")
sys.stderr.reconfigure(encoding="utf-8", errors="replace")
except Exception:
pass
sql_path = Path(sys.argv[1]).expanduser().resolve() if len(sys.argv) > 1 else reso
if not sql_path.is_file():
msg = f"SQL file not found: {sql_path}"
print(msg, file=sys.stderr, flush=True)
LAST_RUN_LOG.write_text(msg + "\n", encoding="utf-8")
raise SystemExit(1)
db_path = sql_path.parent / f"{sql_path.stem}.db"
lines: list[str] = []
def log(msg: str) -> None:
_emit(msg, lines)
log("Lab16.py started.")
try:
run_sql_file(sql_path, db_path, log)
log("Lab16.py finished OK.")
log(f"(Copy of this output: {LAST_RUN_LOG})")
finally:
LAST_RUN_LOG.write_text("\n".join(lines) + "\n", encoding="utf-8")
if __name__ == "__main__":
try:
main()
except SystemExit:
raise
except Exception:
tb = traceback.format_exc()
try:
sys.stderr.write(tb)
sys.stderr.flush()
except Exception:
pass
LAST_RUN_LOG.write_text(tb, encoding="utf-8")
raise SystemExit(1)